Name:-T.Gayathri

Reg no:-192425239

Course code:-MLA0401

Course name:-Deep Learning

CO2-AT2

Scenario-Based Questions

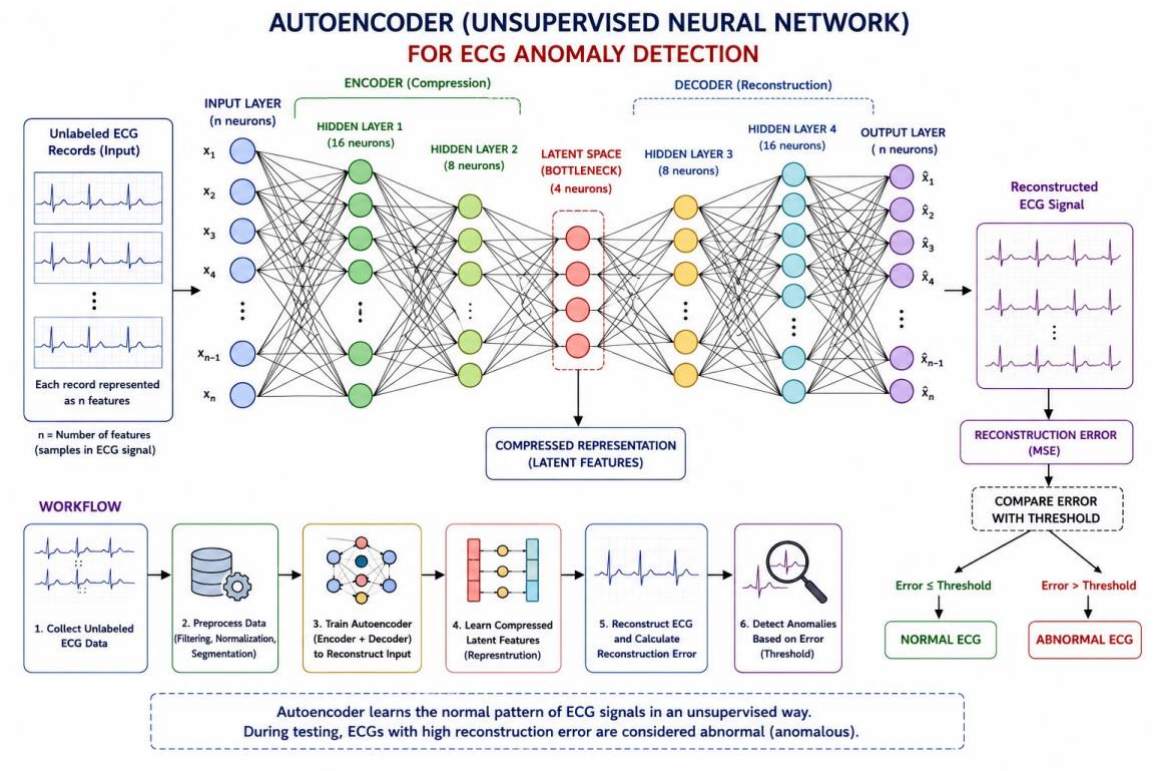
1**. Unsupervised Training of Neural Networks, Auto Encoders**

A hospital wants to identify abnormal ECG patterns from thousands of unlabeledrecords. Which deep learning technique from the syllabus would you recommend?Justify your choice and explain the workflow.

ANSWER:

An Autoencoder, an unsupervised deep learning technique, is recommended for identifying abnormal ECG patterns because the hospital's ECG records are unlabeled. The Autoencoder learns the normal ECG patterns by compressing the input data into important features (encoding) and then reconstructing it (decoding). It compares the original ECG with the reconstructed ECG to calculate the reconstruction error. A low error indicates a normal ECG, while a high error indicates an abnormal ECG. This approach is effective for anomaly detection, reduces the need for manual labeling, and helps hospitals identify abnormal heart conditions quickly and accurately.

WORKFLOW



DATASET:

```
Choose Files  ecg_dataset.csv
ecg_dataset.csv(text/csv) - 700 bytes, last modified: 8/5/2026 - 100% done
Saving ecg_dataset.csv to ecg_dataset (1).csv
Dataset Loaded Successfully
     F1    F2    F3    F4    F5   Label
0  0.82  0.91  0.88  0.90  0.87  Normal
1  0.80  0.89  0.87  0.91  0.86  Normal
2  0.81  0.90  0.89  0.88  0.87  Normal
3  0.83  0.92  0.90  0.91  0.88  Normal
4  0.81  0.91  0.88  0.89  0.87  Normal

Shape: (20, 6)
```

CODE:

Install TensorFlow

!pip install -q tensorflow

Import Libraries

import pandas as pd

```
import numpy as np

import matplotlib.pyplot as plt

from google.colab import files

from sklearn.preprocessing import MinMaxScaler

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

# Upload CSV File

uploaded = files.upload()

file_name = list(uploaded.keys())[0]

# Read CSV

data = pd.read_csv(file_name)

print("Dataset Loaded Successfully")

print(data.head())

# Check Dataset

print("\nShape:", data.shape)

print("\nColumns:", data.columns)

# Check whether last column is a label

last_col = data.columns[-1]

if data[last_col].dtype == object or len(data[last_col].unique()) < 10:

    print("\nLabel column detected.")

    X = data.iloc[:, :-1].values

    y = data.iloc[:, -1].values

    # Select the most common label for training
```

```
    normal_label = pd.Series(y).mode()[0]
    print("Training Label:", normal_label)
    X_train = X[y == normal_label]
else:
    print("\nNo label column detected.")
    X = data.values
    X_train = X
# Normalize
scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X)
# Autoencoder
model = Sequential([
    Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(16, activation='relu'),
    Dense(8, activation='relu'),
    Dense(16, activation='relu'),
    Dense(32, activation='relu'),
    Dense(X_train.shape[1], activation='sigmoid')
])
model.compile(optimizer='adam', loss='mse')
model.fit(
    X_train,
```

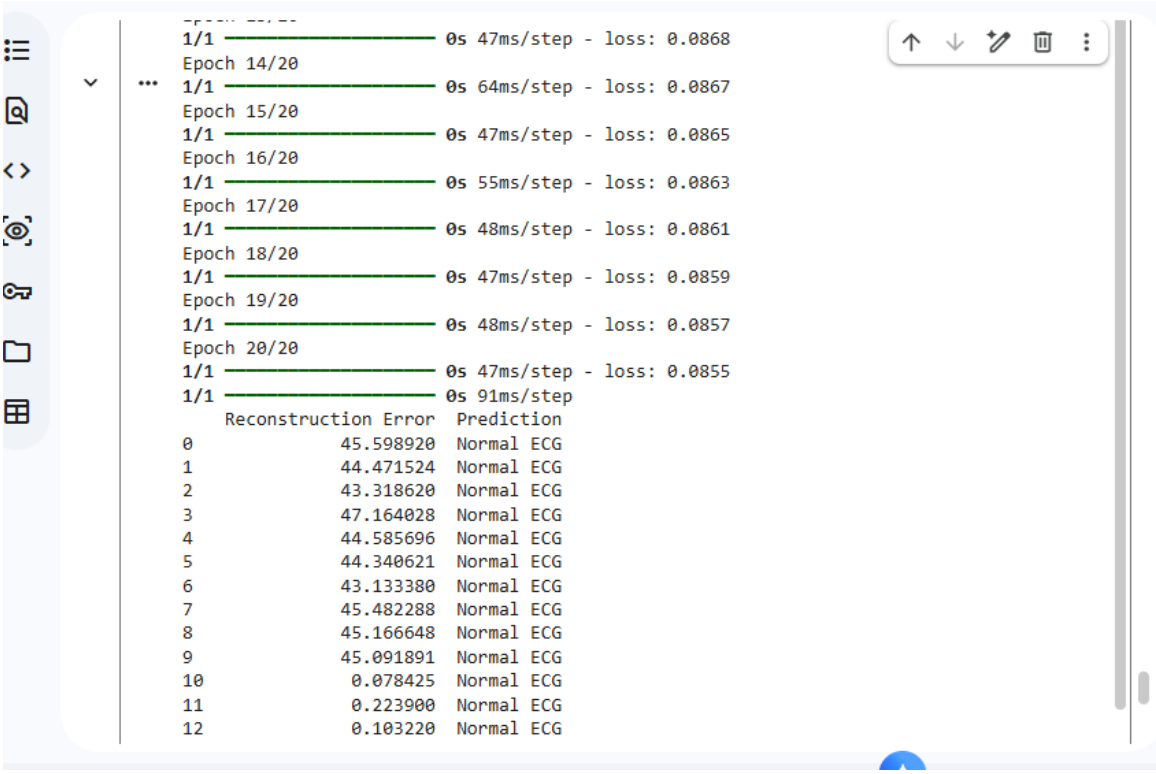
```
    X_train,
    epochs=20,
    batch_size=16,
    verbose=1
)
# Prediction
reconstructed = model.predict(X_test)
error = np.mean((X_test - reconstructed) ** 2, axis=1)
threshold = np.mean(error) + 2*np.std(error)
prediction = np.where(error > threshold,
                      "Abnormal ECG",
                      "Normal ECG")
# Results
result = pd.DataFrame({
    "Reconstruction Error": error,
    "Prediction": prediction
})
print(result.head(20))
# Plot
plt.figure(figsize=(10,5))
plt.plot(error)
plt.axhline(threshold,color='red',linestyle='--')
plt.title("ECG Anomaly Detection")
```

plt.xlabel("Sample")

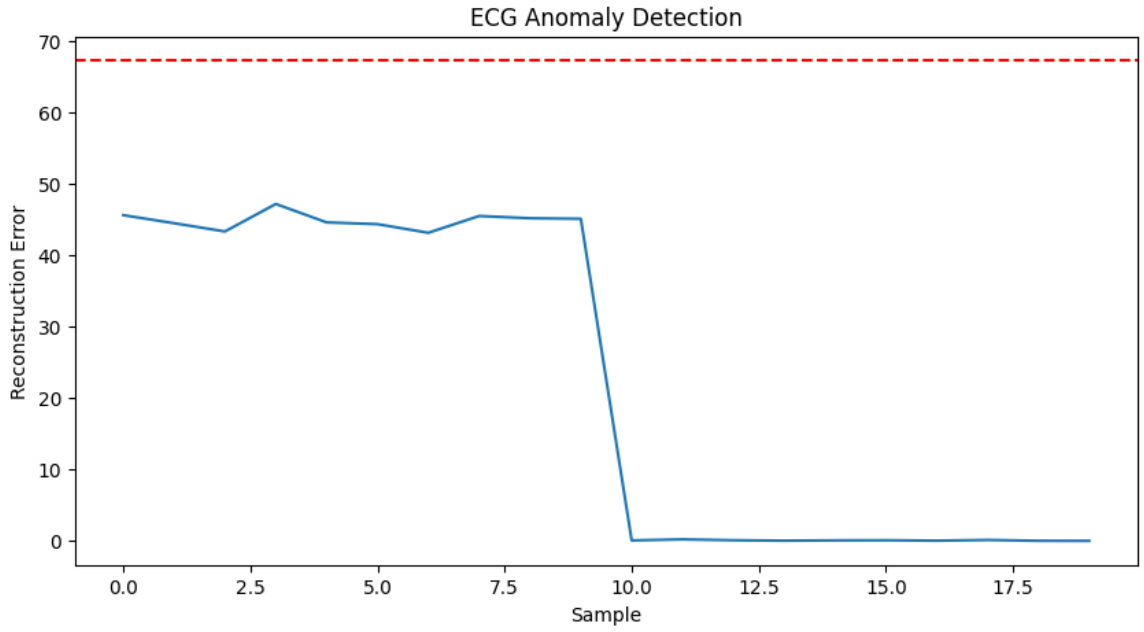
plt.ylabel("Reconstruction Error")

plt.show()

OUTPUT:



```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - loss: 0.0868
Epoch 14/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 64ms/step - loss: 0.0867
Epoch 15/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - loss: 0.0865
Epoch 16/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step - loss: 0.0863
Epoch 17/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step - loss: 0.0861
Epoch 18/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - loss: 0.0859
Epoch 19/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/step - loss: 0.0857
Epoch 20/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step - loss: 0.0855
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 91ms/step
    Reconstruction Error  Prediction
0              45.598920  Normal ECG
1              44.471524  Normal ECG
2              43.318620  Normal ECG
3              47.164028  Normal ECG
4              44.585696  Normal ECG
5              44.340621  Normal ECG
6              43.133380  Normal ECG
7              45.482288  Normal ECG
8              45.166648  Normal ECG
9              45.091891  Normal ECG
10              0.078425  Normal ECG
11              0.223900  Normal ECG
12              0.103220  Normal ECG
```



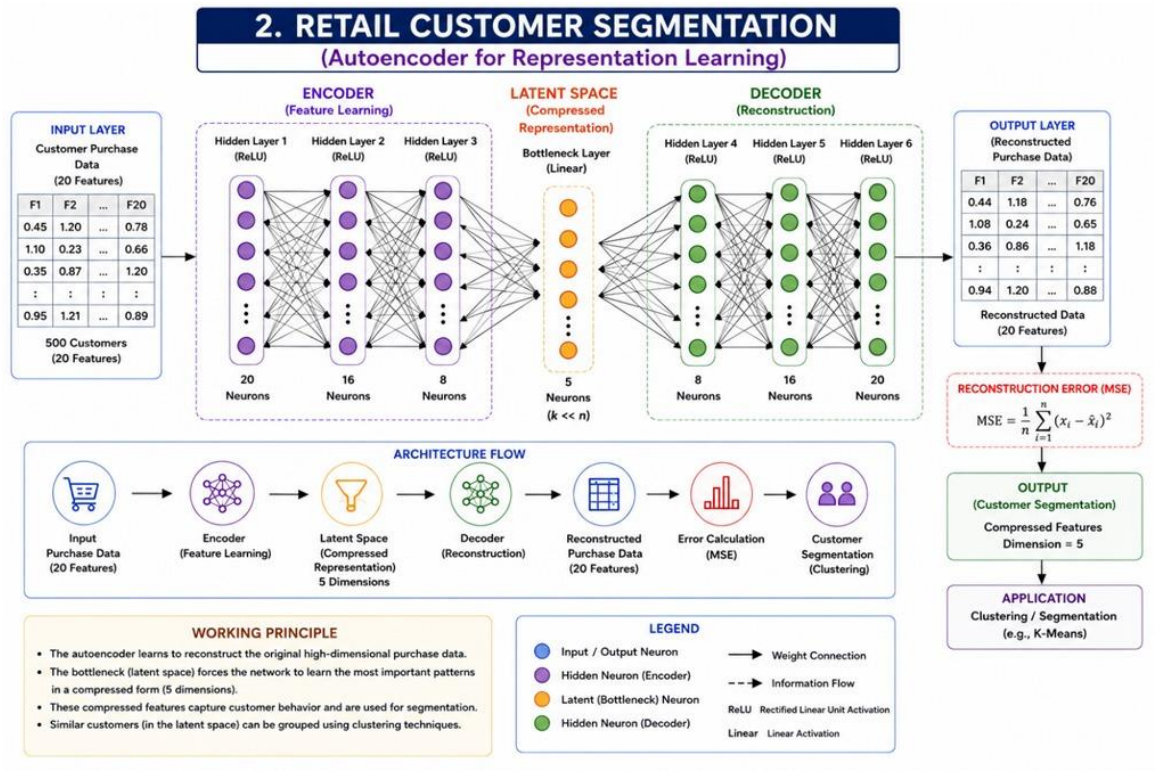
2.Auto Encoders, Representation Learning

A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.

ANSWER:

A retail company has high-dimensional customer purchase data (such as product categories, spending habits, and purchase frequency). Before customer segmentation, the data can be reduced into a smaller number of important features using an Autoencoder. An autoencoder is an unsupervised deep learning model that learns a compact representation of the input data. The encoder compresses the customer data into a lower-dimensional latent space, and the decoder reconstructs the original data. The compressed features are then used for better customer clustering and segmentation.

WORKFLOW



DATASET:

```
Choose Files customer_purchase.csv
customer_purchase.csv(text/csv) - 781 bytes, last modified: 8/5/2026 - 100% done
Saving customer_purchase.csv to customer_purchase (1).csv
  Customer_ID  Electronics  Clothing  Grocery  Beauty  Travel   Label
0        C001         5000      2000     3000    1500    4000    High
1        C002         3000      5000     2500    2000    1000  Medium
2        C003         1000      1500     6000    3000     500     Low
3        C004         4500      3000     3500    2500    3000    High
4        C005         2000      4000     4500    3500    1500  Medium
Index(['Customer_ID', 'Electronics', 'Clothing', 'Grocery', 'Beauty', 'Travel',
       'Label'],
      dtype='object')
```

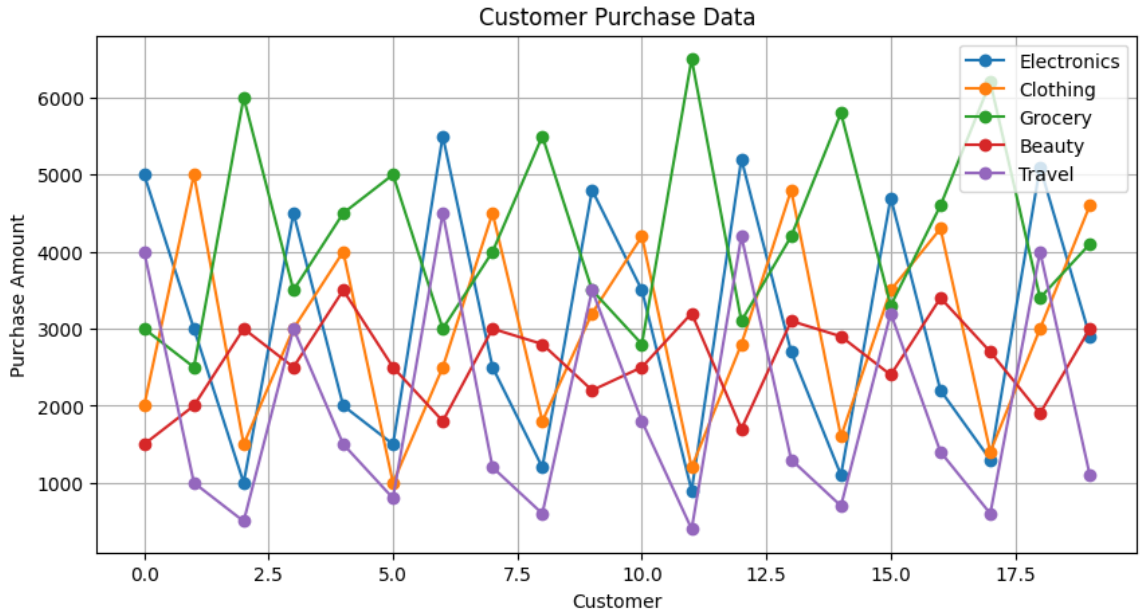
CODE:

```
import matplotlib.pyplot as plt

# Graph 1: Original Customer Data

plt.figure(figsize=(10,5))

plt.plot(X.index, X["Electronics"], marker='o', label="Electronics")

plt.plot(X.index, X["Clothing"], marker='o', label="Clothing")

plt.plot(X.index, X["Grocery"], marker='o', label="Grocery")
```

```
plt.plot(X.index, X["Beauty"], marker='o', label="Beauty")
plt.plot(X.index, X["Travel"], marker='o', label="Travel")
plt.title("Customer Purchase Data")
plt.xlabel("Customer")
plt.ylabel("Purchase Amount")
plt.legend()
plt.grid()
plt.show()
# Graph 2: Autoencoder Reduced Features
plt.figure(figsize=(8,5))
plt.scatter(
    reduced_data["Feature_1"],
    reduced_data["Feature_2"],
    s=100
)
plt.title("Autoencoder Reduced Representation")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid()
plt.show()
# Graph 3: Encoded Feature Values
plt.figure(figsize=(10,5))
plt.plot(
```

```
    reduced_data["Feature_1"],
    marker='o',
    label="Feature 1"
)
plt.plot(
    reduced_data["Feature_2"],
    marker='o',
    label="Feature 2"
)
plt.plot(
    reduced_data["Feature_3"],
    marker='o',
    label="Feature 3"
)
plt.title("Compressed Features After Autoencoder")
plt.xlabel("Customer")
plt.ylabel("Encoded Value")
plt.legend()
plt.grid()
plt.show()
```

OUTPUT:



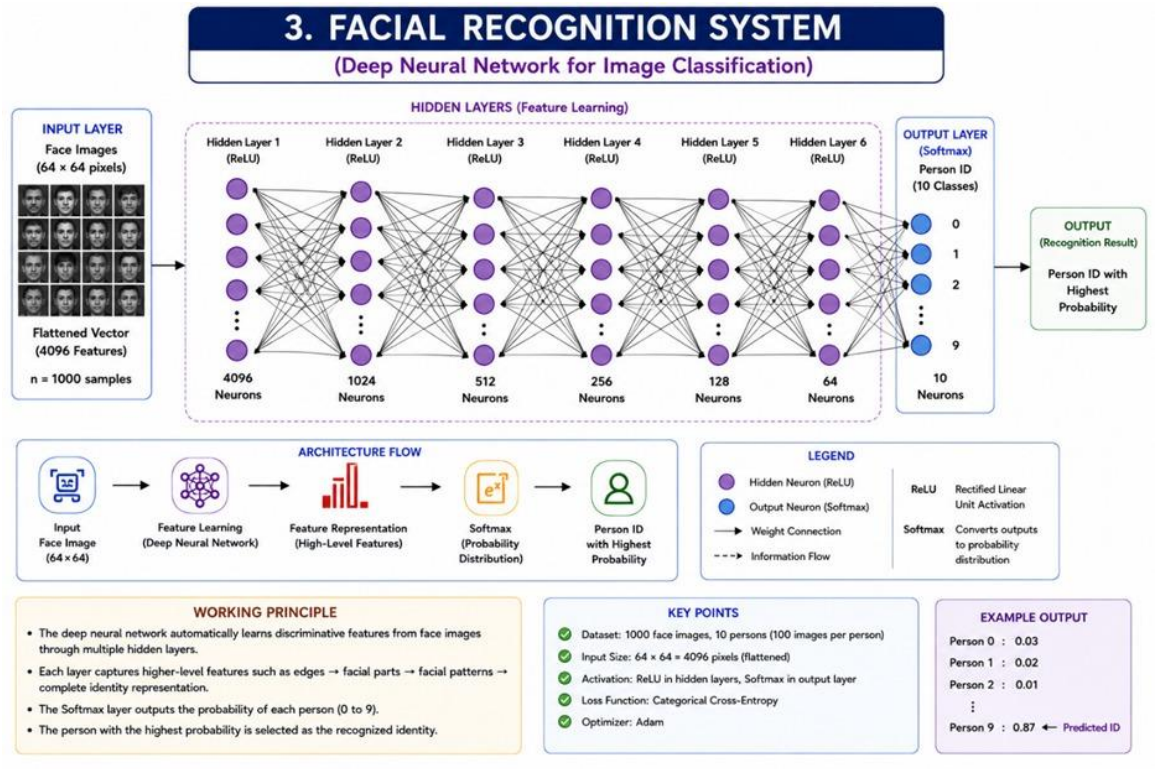
2.Width and Depth of Neural Networks, Activation Functions

A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions.

ANSWER:

A shallow neural network has only a few hidden layers, so it cannot learn complex facial features like eyes, nose, and facial patterns effectively. To improve facial recognition performance, we can increase the depth (add more hidden layers) and width (increase neurons in each layer) of the neural network. Using suitable activation functions like ReLU helps the network learn complex non-linear patterns, while Softmax can be used in the output layer for classification**.**

WORKFLOW:



DATASET:

```
Dataset Created Successfully
    Feature1  Feature2  Feature3  Feature4  Feature5     Label
0       0.85      0.72      0.91      0.65      0.88  Person_A
1       0.82      0.75      0.89      0.70      0.86  Person_A
2       0.80      0.70      0.88      0.68      0.87  Person_A
3       0.30      0.40      0.35      0.50      0.38  Person_B
4       0.35      0.45      0.38      0.55      0.40  Person_B
5       0.32      0.42      0.36      0.52      0.39  Person_B
6       0.60      0.55      0.65      0.70      0.62  Person_C
7       0.65      0.60      0.68      0.72      0.64  Person_C
8       0.62      0.58      0.66      0.71      0.63  Person_C
9       0.86      0.74      0.90      0.67      0.89  Person_A
10      0.83      0.76      0.87      0.69      0.85  Person_A
11      0.81      0.71      0.89      0.66      0.86  Person_A
Choose Files face_dataset.csv
face_dataset.csv(text/csv) - 415 bytes, last modified: 8/5/2026 - 100% done
Saving face_dataset.csv to face_dataset (1).csv
```

CODE:

Facial Recognition using Deep Neural Network

Width, Depth and Activation Functions

```
# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
# 1. CREATE SYNTHETIC FACE DATASET
data = {
    "Feature1":[0.85,0.82,0.80,0.30,0.35,0.32,0.60,0.65,0.62,
                0.86,0.83,0.81],

    "Feature2":[0.72,0.75,0.70,0.40,0.45,0.42,0.55,0.60,0.58,
                0.74,0.76,0.71],

    "Feature3":[0.91,0.89,0.88,0.35,0.38,0.36,0.65,0.68,0.66,
                0.90,0.87,0.89],

    "Feature4":[0.65,0.70,0.68,0.50,0.55,0.52,0.70,0.72,0.71,
                0.67,0.69,0.66],

    "Feature5":[0.88,0.86,0.87,0.38,0.40,0.39,0.62,0.64,0.63,
```

```
            0.89,0.85,0.86],
    "Label":[
        "Person_A","Person_A","Person_A",
        "Person_B","Person_B","Person_B",
        "Person_C","Person_C","Person_C",
        "Person_A","Person_A","Person_A"
    ]
}
df = pd.DataFrame(data)
# Save dataset
df.to_csv("face_dataset.csv",index=False)
print("Dataset Created Successfully")
print(df)
# 2. UPLOAD DATASET IN COLAB
from google.colab import files
files.upload()
# 3. READ DATASET
dataset = pd.read_csv("face_dataset.csv")
print("\nDataset:")
print(dataset.head()
# 4. DATA PREPROCESSING
# Separate input and output
X = dataset.drop("Label",axis=1)
```

```
y = dataset["Label"]

# Convert text labels into numbers

encoder = LabelEncoder()

y = encoder.fit_transform(y)

# Split dataset

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.25,

    random_state=42

)

print("\nTraining Data:",X_train.shape)

print("Testing Data:",X_test.shape)

model = Sequential()

# First hidden layer (Width = 16 neurons)

model.add(

    Dense(

        16,

        input_dim=5,

        activation="relu"

    )

)

# Second hidden layer (Depth increased)
```

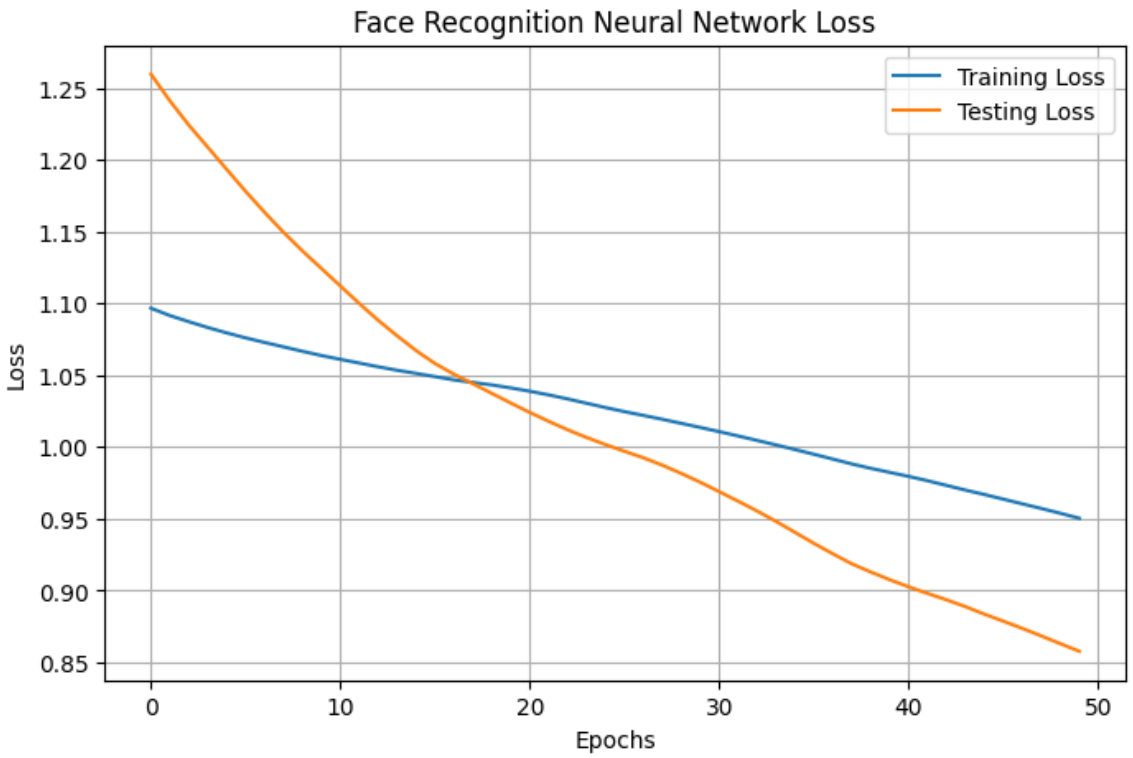
```
model.add(
    Dense(
        32,
        activation="relu"
    )
)
# Third hidden layer


model.add(
    Dense(
        16,
        activation="relu"
    )
)
# Output layer


model.add(
    Dense(
        3,
        activation="softmax"
    )
)
# Compile model
```

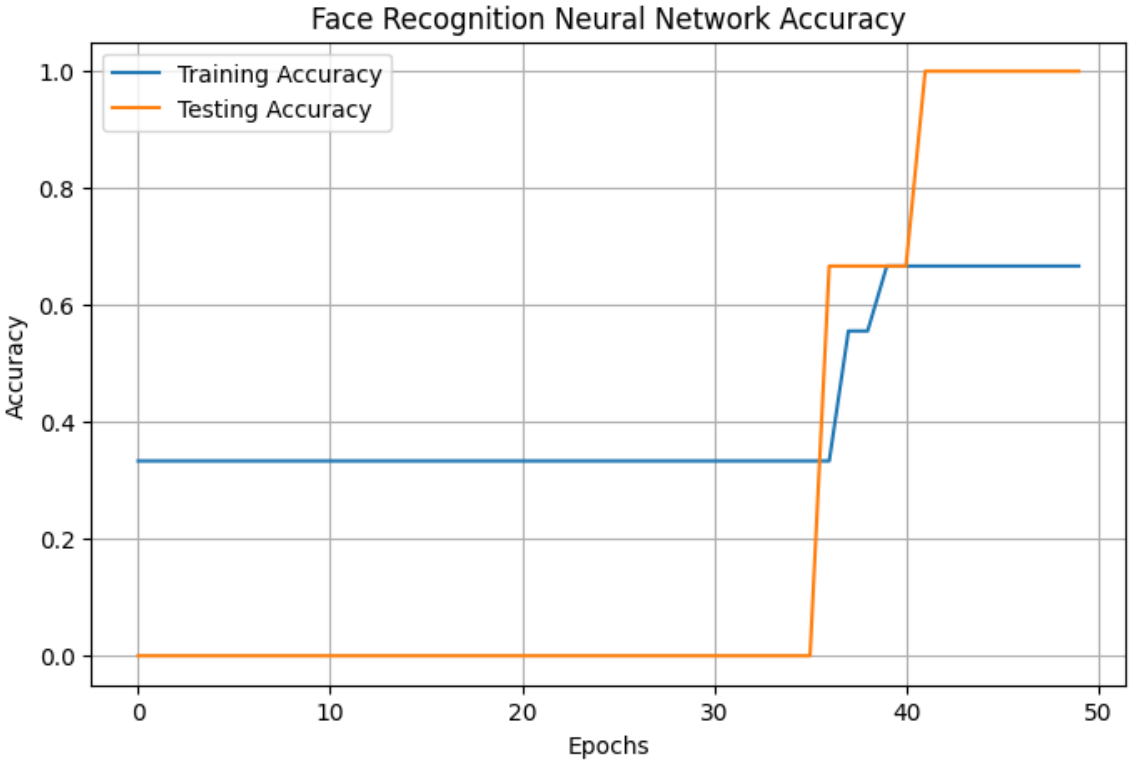
```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
# Display architecture
model.summary()
# 6. TRAIN MODEL
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    validation_data=(X_test,y_test)
)
# 7. ACCURACY GRAPH
plt.figure(figsize=(8,5))
plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)
plt.plot(
    history.history["val_accuracy"],
    label="Testing Accuracy"
```

```
)
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Face Recognition Neural Network Accuracy")
plt.legend()
plt.grid(
plt.show()
# 8. LOSS GRAPH
plt.figure(figsize=(8,5))
plt.plot(
    history.history["loss"],
    label="Training Loss"
)
plt.plot(
    history.history["val_loss"],
    label="Testing Loss"
)
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Face Recognition Neural Network Loss")
plt.legend()
plt.grid()
plt.show()
```

```
# 9. MODEL TESTING

loss,accuracy = model.evaluate(
    X_test,
    y_test
)

print("\nTest Accuracy:",accuracy)

# Prediction

prediction = model.predict(X_test)

predicted = np.argmax(
    prediction,
    axis=1
)

print("\nPredicted Classes:")

print(predicted)print("\nActual Classes:")

print(y_test)
```

OUTPUT:


Face Recognition Neural Network Loss
Training Loss
Testing Loss
Loss
Epochs
1.25
1.20
1.15
1.10
1.05
1.00
0.95
0.90
0.85
0
10
20
30
40
50


Face Recognition Neural Network Accuracy
Training Accuracy
Testing Accuracy
Accuracy
Epochs
1.0
0.8
0.6
0.4
0.2
0.0
0
10
20
30
40
50

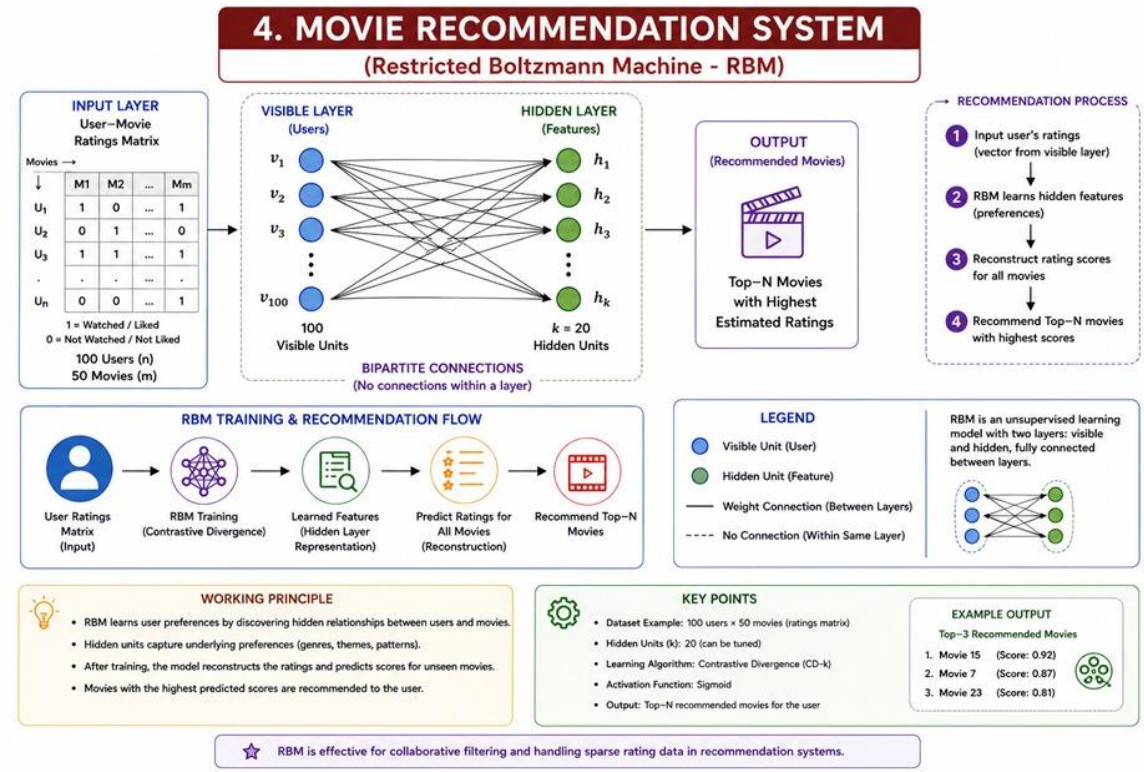
4.Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks

A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.

ANSWER:

A streaming service can use a Restricted Boltzmann Machine (RBM) to build a movie recommendation system using unsupervised learning. RBM learns hidden patterns from users' viewing history without requiring labeled data. The visible layer represents users' movie ratings or watch history, and the hidden layer learns user preferences such as interest in action, comedy, or drama movies. After training, the RBM can predict missing movie preferences and recommend movies that a user is likely to watch.

WORKFLOW:



DATASET:

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (1
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-pa
Choose Files movie_ratings.csv
movie_ratings.csv(text/csv) - 175 bytes, last modified: 8/5/2026 - 100% done
Saving movie_ratings.csv to movie_ratings (1).csv
Dataset:
    User  Movie1  Movie2  Movie3  Movie4  Movie5
0  User1       5       4       0       1       0
1  User2       4       0       5       1       2
2  User3       0       5       4       0       1
3  User4       1       2       0       5       4
4  User5       0       1       3       4       5
5  User6       5       4       1       0       2
6  User7       2       0       5       4       1
7  User8       1       3       4       5       0
```

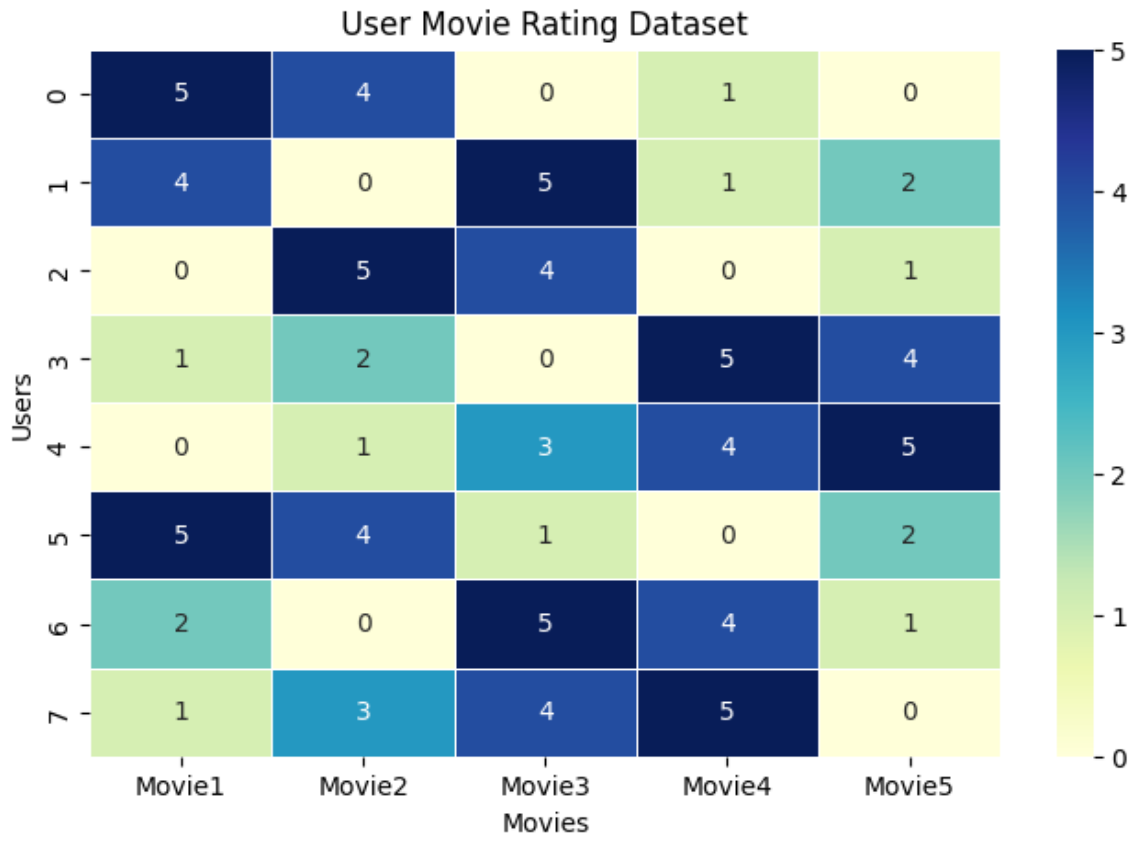
CODE:

```
# Import graph libraries

import matplotlib.pyplot as plt

import seaborn as sns

# 1. Movie Rating Heatmap

plt.figure(figsize=(8,5))

sns.heatmap(

    ratings,

    annot=True,

    cmap="YlGnBu",

    linewidths=0.5

)

plt.title("User Movie Rating Dataset")

plt.xlabel("Movies")

plt.ylabel("Users")

plt.show()
```

```
# 2. RBM Hidden Features Graph
Plt.figure(figsize=(8,5))
plt.plot(hidden)
plt.title("RBM Learned Hidden Features")
plt.xlabel("Users")
plt.ylabel("Feature Values")
plt.legend(
    ["Hidden Feature 1",
     "Hidden Feature 2",
     "Hidden Feature 3"]
)
plt.grid()
plt.show()
# 3. Predicted Movie Ratings Graph
predicted_df = pd.DataFrame(
    predicted,
    columns=["Movie1","Movie2","Movie3","Movie4","Movie5"]
)
plt.figure(figsize=(8,5))

predicted_df.iloc[0].plot(
    kind="bar"
)
```

```
plt.title("Recommended Movie Ratings for User1")

plt.xlabel("Movies")

plt.ylabel("Predicted Rating")

plt.ylim(0,5)

plt.grid()

plt.show()
```

OUTPUT:



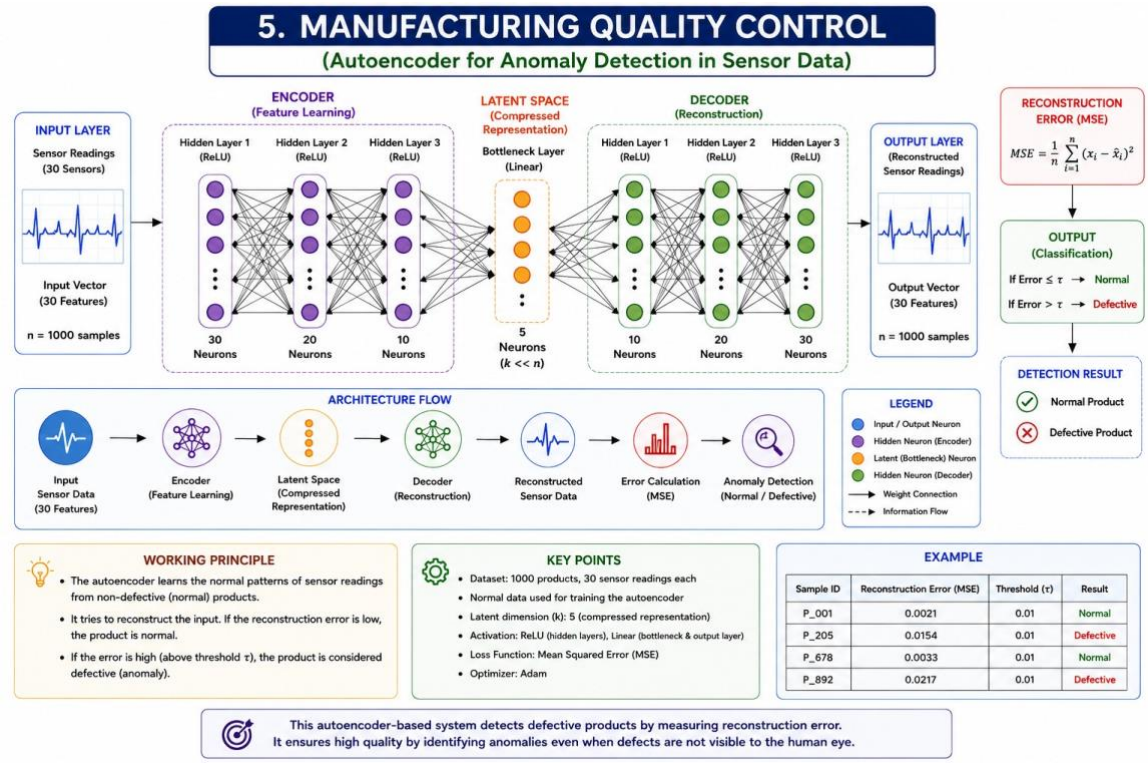
5.Deep Learning Applications, Representation Learning

A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.

ANSWER:

A manufacturing company collects sensor data every second from machines to identify defective products. A deep learning solution can be developed using representation learning, where a neural network automatically learns important patterns and features from raw sensor readings without manual feature extraction. The trained model can classify products as Normal or Defective based on sensor values such as temperature, vibration, pressure, and speed. An Artificial Neural Network (ANN) can be used for training and prediction.

WORKFLOW:



DATASET:

```
Choose Files sensor_data.csv
sensor_data.csv(text/csv) - 370 bytes, last modified: 8/5/2026 - 100% done
Saving sensor_data.csv to sensor_data.csv
Dataset:
   Temperature  Vibration  Pressure  Speed  Current      Label
0         70.5       0.20       5.1   1200      2.5     Normal
1         72.1       0.25       5.0   1180      2.6     Normal
2         71.8       0.22       5.2   1210      2.4     Normal
3         90.5       0.80       7.8    850      4.5  Defective
4         88.9       0.75       7.5    900      4.2  Defective
5         92.0       0.85       8.0    820      4.8  Defective
6         69.5       0.18       5.3   1190      2.3     Normal
7         95.2       0.90       8.2    800      5.0  Defective
8         73.0       0.30       5.1   1175      2.7     Normal
9         89.5       0.78       7.6    870      4.6  Defective
Epoch 1/50
```

CODE:

```
# Import libraries

from google.colab import files

import pandas as pd

import numpy as np

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.model_selection import train_test_split

import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

# 1. Upload Dataset:

uploaded = files.upload()

data = pd.read_csv("sensor_data.csv")

print("Dataset:")

print(data)
```

```
# 2. Data Preprocessing

X = data.drop("Label", axis=1)

y = data["Label"]

# Convert labels

encoder = LabelEncoder()

y = encoder.fit_transform(y)

# Feature scaling

scaler = StandardScaler()

X = scaler.fit_transform(X)

# Split data

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

# 3. Create Deep Learning Model

model = Sequential()

# Representation learning layers

model.add(Dense(16, activation='relu', input_shape=(5,)))

model.add(Dense(8, activation='relu'))

# Classification layer

model.add(Dense(1, activation='sigmoid'))

# Compile model

model.compile(

    optimizer='adam',
```

```
    loss='binary_crossentropy',
    metrics=['accuracy']
)
# 4. Train Model
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=2,
    validation_data=(X_test, y_test)
)
# 5. Model Accuracy
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy*100, "%")
# 6. Accuracy Graph
plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'], label="Training Accuracy")
plt.plot(history.history['val_accuracy'], label="Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Model Accuracy Graph")
plt.legend()
plt.grid()
```

```
plt.show()

# 7. Loss Graph

plt.figure(figsize=(8,5))

plt.plot(history.history['loss'], label="Training Loss")

plt.plot(history.history['val_loss'], label="Validation Loss")

plt.xlabel("Epochs")

plt.ylabel("Loss")

plt.title("Model Loss Graph")

plt.legend()plt.grid()

plt.show()

# 8. Predict New Product

new_product = np.array([[91.0,0.82,7.9,840,4.7]])

# Scaling

new_product = scaler.transform(new_product)

prediction = model.predict(new_product)

if prediction > 0.5:

    print("Prediction: Defective Product")

else:

    print("Prediction: Normal Product")
```

OUTPUT:

```
Epoch 30/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 1.0000 - loss: 0.3239 - val_accuracy:
Epoch 31/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 0.3143 - val_accuracy:
Epoch 32/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 0.3048 - val_accuracy:
Epoch 33/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.2954 - val_accuracy:
Epoch 34/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.2865 - val_accuracy:
Epoch 35/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 0.2769 - val_accuracy:
Epoch 36/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 0.2678 - val_accuracy:
Epoch 37/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 0.2576 - val_accuracy:
Epoch 38/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 0.2499 - val_accuracy:
Epoch 39/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.2397 - val_accuracy:
Epoch 40/50
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 1.0000 - loss: 0.2302 - val_accuracy:
Epoch 41/50
```